Daskalos Smalltalk Cheatsheet

Important rules to remember:

1. Methods are always called on a receiver. You cannot call a method without telling which object should execute the method.
2. Statements (commands, if you want) are always terminated by **.** (dot).
3. Strings are written between two apostrophes, e.g. **'Hello World'**
4. Do not use quotes for strings, quotes are used for comments!
5. Single characters (chars) are escaped by the $ character, e.g. **$A**, **$5**, **$!** – if you want the dollar character, use **$$**, if you want space, just **$** is used.
6. Everything is an object.

Special keywords

- **nil** undefined object, default value of a variable which has no explicit value assigned

- **true** boolean value „true“
- **false** boolean values „false“

- **super** reference to the parent class (superclass) used to invoke parent class's implementation of a method
- **self** reference to itself, it is used to invoke methods of the same object (you will need it when implementing a method in a class that needs to call another method of the same class)

Method execution

Multiple methods can be called on the same object in one statement. Their execution order is evaluated left to right until the end of statement separator is found (.). If you enclose a part of the statement inside parentheses (), the content of parentheses is evaluated first.

Messages (method calls) are evaluated in the following order:

1. **unary messages** – methods without parameters
2. **binary messages** – methods with one argument, where the method selector doesn't contain a colon (:). Arithmetic operators, comparison operators etc.
3. **keyword messages** – methods taking one or more parameters and use a word with colon (:) before each parameter.

Remember that when you chain multiple method calls in one statement, each method call is applied to the result of the previous call:

5 factorial squared * 2 has the same meaning as **((5 factorial) squared) * 2.** First the factorial of 5 is calculated, the result is squared and finally the result is multiplied by 2.

Creating new objects (instances)

Method **new** is used for creating new objects in Smalltalk/Daskalos. This method is only available in the class (type) itself; it cannot be called on instances. Result of this call is a reference to the newly created object, which must be in some way stored (put inside a collection, assigned to a variable, etc.), otherwise the object cannot be accessed and is soon destroyed by the garbage collector.

Let's suppose we have a class called Car. A new instance of this class can be created like this:

a := Car new.

This statement means that the Car class is asked for its new instance and this instance is then assigned the name **a** (basically it is put inside the variable of this name).

Blocks

Blocks are enclosed between square parentheses – [and]. Blocks are used to create inner functions inside your code. To execute the block and get the result of execution, **value** message is used.

[1 + 2 + 3] value.	returns 6

Blocks can accept parameters. Those are defined in the block header, which is written at the beginning of the block and is separated from the body of the block (the statements that do something) by the **|** character. Each parameter definition starts with colon character and multiple parameters can be defined, separated by space. When executing such a block, each input value must be provided after a separate **value:** message.

[:x | x + 3] value: 5	returns 8
[:x :y | x + y] value: 5 value: 6	return 11

If a block contains multiple statements in its body, the result of evaluation of the last statement is returned as the return value of the whole block. Blocks are often used as parameters for other methods, including and: and or: logic operations, or for many operations defined on various collections.

Conditions

Evaluation of a condition results in a value of **true** or **false**, which are instance of the True and False classes. More complex conditions and be defined using **and:** and **or:** messages. Those messages expect blocks defining the second part of the condition.

x > 10 and: [x < 20]

y < 10 or: [y > 20]

In Daskalos we can also use the & and | operators instead the and: and or: messages. The conditions are then written differently. The conditions above would like this:

(x > 10) & (x < 20)

(y < 10) | (y > 20)

Block of code can be executed based on the result of evaluation of a condition. Although the function and purpose is the same as the if/else conditional statement in other programming languages, we have to use one of the following 4 methods for this in small talk – **ifTrue:**, **ifFalse:**, **ifTrue: ifFalse:** and **ifFalse: ifTrue:**. This way we can achieve any needed combination.

(x > 10) ifTrue: [y := x+10]	if x > 10, then y is assigned x + 10
(x > 10) ifFalse: [y := x+10]	if NOT x > 10, then y is assigned x + 10
(x>10) ifTrue: [y := x+10] ifFalse: [z := x-5]	if x>10, then y is assigned x + 10, otherwise z is assigned x - 5
(x>10) ifFalse: [y := x+10] ifTrue: [z := x-5]	if NOT x>10, then y is assigned x + 10, otherwise z is assigned x - 5

Collections

Collections are special types (classes) allowing grouping of objects in their instances. Every collection type is basically a description of a certain type of a container for grouping objects. Each collection class has different attributes and its behavior may be different from other collection types. In Daskalos we primarily use 3 main collection types: Set, Bag and List.

Those collections can be divided into two groups, based on the way we can access their items:

- Collections without known order of items – items inside the collection cannot be accessed directly.
    - **Set**
        - doesn't allow copies of the same object – even if the same object is added multiple times, only one copy present inside the collection
    - **Bag**
        - does allow copies of the same object – if the same object is added multiple times, multiple copies are present inside the collection

- Collections with known order of items – items inside the collection can be accessed directly, because we known their positions
    - **List**
        - does allow copies of the same object and each item (object) in the collection can be directly accessed using its index (position). Numbering of items starts with **1**.

Operations with collections

Collections support many operations that are common to most of the collection types. Below is the list of the most important ones that may be useful to you for your semestral project.

- **add:**
    - add an item into a collection, can be chained in cascade
    - e.g. **X add: o1.**
    - **X add: o1; add: o2; add: o3.**
- **addAll:**
    - add all items from another collection
    - e.g. **X addAll: #(1 2 3 4 5 6)**
- **remove:**
    - removes item which was passed as method's argument from collection
    - e.g. **X remove: o1.**

There are also important operations that provide information about a collection:

- **size**
    - finds how many items are in the collection
    - e.g. **X size**
- **isEmpty**
    - finds whether the collection is empty (empty = true)
    - e.g. **X isEmpty**
- **includes:**
    - finds whether the collection includes an item passed as method's parameter
    - e.g. **X includes: o1;**
- **contains:**
    - finds whether the collection contains items satisfying the condition (returns true/false)
    - e.g. **X contains: [:a | a >10]**

Objects can be selected based on whether they do or do not satisfy a certain defined logic condition. The difference between **select:/reject:** and **detect:** is the way in which the result is provided. For **select:** and **reject:** the result is provided as a collection of objects, even if there is only one object satisfying the condition (i.e. a collection containing one object is returned). On the other hand **detect:** returns directly the first object that satisfies the condition, so the result can be immediately used and it is not necessary to unwrap it from the collection.

- select:
  - selects all the items that satisfy the condition provided in the block
  - e.g **X select : [:a | a price > 1000]**
- detect:
  - returns the first item satisfying condition provided in the block
  - e.g X **detect : [:a | a price > 1000]**
- reject:
  - exact opposite to select :, selects items not satisfying the condition
  - e.g **X reject : [:a | a price > 1000]**

In a similar way we can also transform objects or gather values of certain attribute from all the objects inside the collection Operation **collect:** is used for this purpose. This operation takes every object in the collection and returns a result of executing operation defined in the block. This way we can for example gather prices of all goods stored in the collection etc.

- **collect:**
  - transforms all items in the collection based on the code in the block
  - e.g. **X collect : [:a | a price]**

IMPORTANT NOTE: select:, **reject:** and **detect:** are operations returning the results stored in a collection. The type of the result collection is the same as the type of the collection the operation is being executed on. E.g., if we select some values from a Set collection, the result collection will again be of the Set type. This must be considered especially when using the **collect:** operation, as **originally unique values** in the origin collection **may lead to non-unique values** in the result. And if the result is again stored as Set, the potential **duplicates will be removed**.

Methods

Methods are defined in class definitions and in the case of Daskalos they are used to work with the data stored in attributes of instances of the class. Unlike in real application development, where methods may return a value (so basically act like a function) or just do a certain operation without returning anything (acting like a procedures), in Daskalos we expect all methods to return a value – their purpose is solely to be a way of implementing dynamic attributes whose values are calculated or in any other way derived from other data already present in the object.

General structure of such a method looks like follows.

```
messageSelector: parameterName

    | variableA variableB variable |

    statement1.
    statement2.
    statement3.
    ^returnStatement.
```

Definition of a method begins by specifying the message selector, i.e. the name of the method. If the method is expected to accept a value as its parameter, the name of the method must end with colon and the parameter name must be provided after it. If the method doesn't need any parameter, only the name of the method is specified (see examples below).

If any temporary variables are needed in the method body, their names must be defined inside a section contained between two | characters. If multiple temporary variables are needed, their names are separated with space.

Since the methods are expected to return a value, we must be able to specify which statement value is to be returned as the return value of the whole method. We do so by starting the statement with the ^ character. This character means that the statement is executed, it's return value obtained and finally this value is then returned outside from the method. The execution of the method ends at that point, even if there are additional statements after this one – they won't be executed.

Example of a method without parameters:

```
priceWithVat
    ^price * 1.21
```

This method would be called simply **object priceWithVat**.

Example of a method with one parameter:

```
squared: numberParameter
    ^(numberParameter ** 2)
```

This method would be called **object squared: 10.**

Example of a method with multiple parameters:

```
pythagorean: valueA otherSide: valueB
    ^(valueA squared + valueB squared) sqrt
```

This method would be called **object pythagorean: 5 otherSide: 6.**